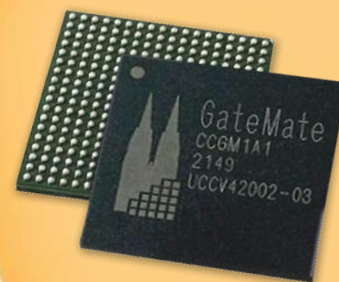


GateMate™ FPGA

PHY Interface for the PCIe Architecture

Vinh Hai Luong, B.Sc.
Research&Development
Cologne Chip AG



DLL Port List

PHY Interface

Direction	Name	Description	Width
Input	phy_clk	PHY-domain clock	1 bit
Input	phy_reset	Async reset from the physical layer	1 bit
Input	phy_linkup	Link-up flag indication from the physical layer (LTSSM in L0 state)	1 bit
Input	phy_rx_data	Receive data stream from the physical layer	DATA_WIDTH
Input	phy_rx_data_k	K-character (control) markers for phy_rx_data	DATA_WIDTH/8
Output	phy_tx_data	Transmit data stream to the physical layer	DATA_WIDTH
Output	phy_tx_data_k	K-character (control) markers for phy_tx_data	DATA_WIDTH/8

* Current supported DATA_WIDTH is 64 and 32 bits

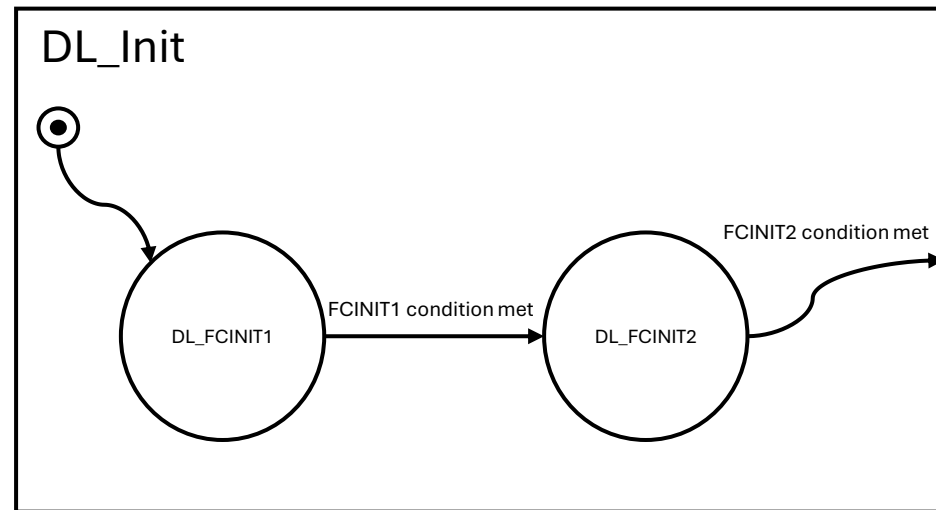
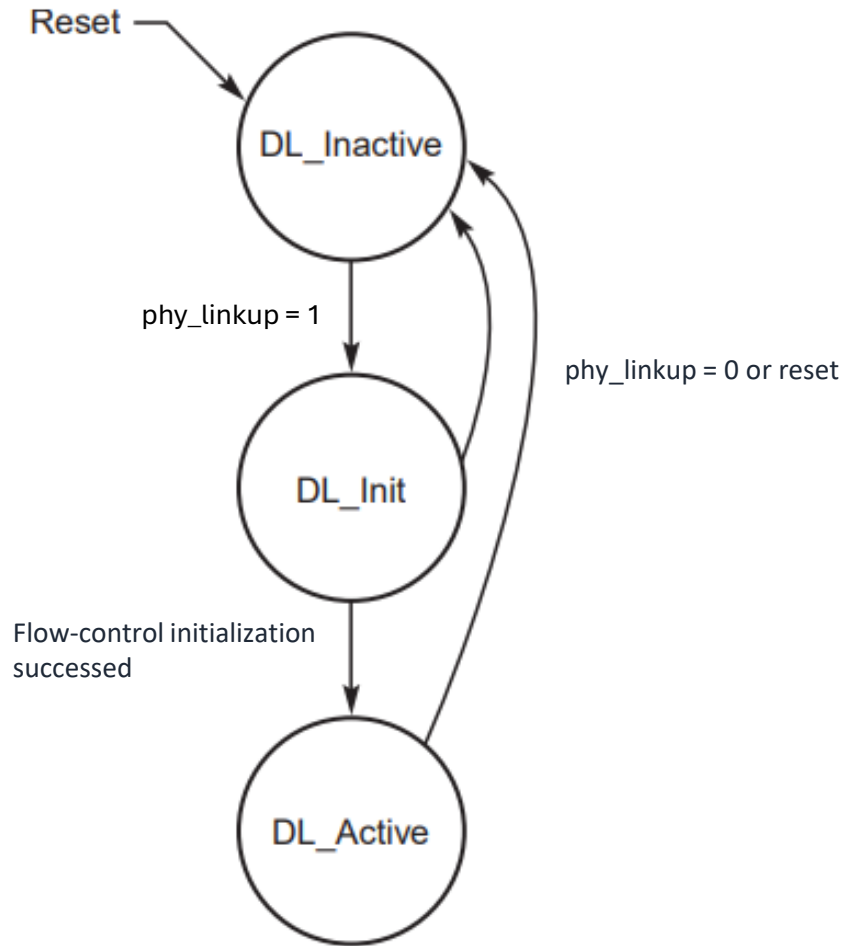
DLL Port List



TL Interface

Direction	Name	Description	Width
Input	tl_tx_data	TLP payload from TL	DATA_WIDTH
Input	tl_tx_tlp_valid tl_tx_tlp_first tl_tx_tlp_last	Valid flag for the TLP payload First batch of the payload from TL Last batch of the payload from TL	1 bit 1 bit 1 bit
Input	tl_tx_hdr_credit tl_tx_data_credit tl_tx_update_type tl_tx_packet_type tl_tx_dllp_valid	Header credits of DLLP from TL Data credits of DLLP from TL Update type of DLLP from TL Packet type of DLLP from TL DLLP tx valid	8 bits 12 bits 2 bits 2 bits 1 bit
Output	tl_rx_data	Extracted TLP payload forwarded to TL	DATA_WIDTH
Output	tl_rx_tlp_valid tl_rx_tlp_first tl_rx_tlp_last	Valid flag for the received TLP payload First batch of the payload to TL Last batch of the payload to TL	1 bit 1 bit 1 bit
Output	tl_rx_hdr_credit tl_rx_data_credit tl_rx_update_type tl_rx_packet_type tl_rx_dllp_valid	Extracted header credits forwarded to TL Extracted data credits forwarded to TL Extracted update type forwarded to TL Extracted packet type forwarded to TL DDLp rx valid	8 bits 12 bits 2 bits 2 bits 1 bit

DLL Init FSM

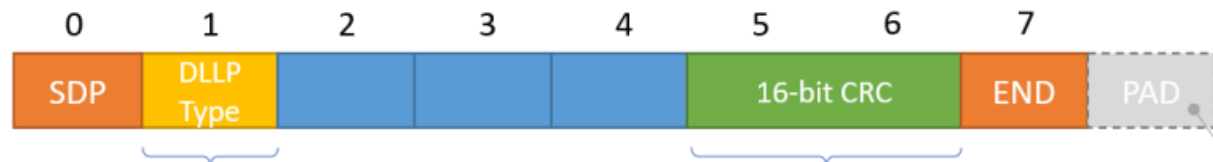


Flow-control initialization



- All TLPs are blocked.
- DL_Init has two states
 - DL_FCINIT1:
 - Transmit and Receive three InitFC1 DLLPs:
 - InitFC1-P
 - InitFC1-NP
 - InitFC1-Cpl
 - Repeat the transmission every 34us if both conditions are not met.
 - Record the FC unit values and move onto DL_FCINIT2.
 - DL_FCINIT2:
 - Transmit and Receive three InitFC2 DLLPs:
 - InitFC2-P
 - InitFC1-NP
 - InitFC1-Cpl
 - Repeat the transmission every 34us if both conditions are not met.
 - Ignore the FC unit values and move onto DL_Active

DLLP structure



0000 0000: ACK
0001 0000: NAK
0010 0000: PM_Enter_L1
0010 0001: PM_Enter_L23
0010 0011: PM_Active State_Request_L1
0010 0100: PM_Request_Ack
0011 0000: Vendor specific
0100 0xxx: InitFC1-P (xxx = virtual channel #)
0101 0xxx: InitFC1-NP
0110 0xxx: InitFC1-Cpl
1100 0xxx: InitFC2-P
1101 0xxx: InitFC2-NP
1110 0xxx: InitFC2-Cpl
1000 0xxx: UpdateFC-P
1001 0xxx: UpdateFC-NP
1010 0xxx: UpdateFC-Cpl

$G(x) = x^{16} + x^{12} + x^3 + x + 1$
Initial value is FFFFh
DLLP bytes fed in LSb first
Bytes of result inverted and bit reversed

*PADs to link width if no new
DLLP or TLP to transmit*

SDP = 0101 1100

END = 1111 1101

PAD = 1111 0111

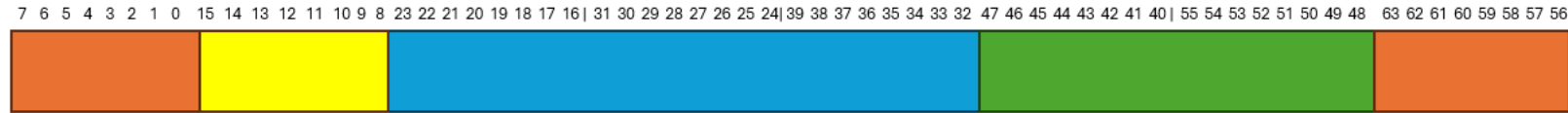
- PM and Vendor DLLPs are not supported in this first version of DLL.
- Padding is not needed for 32-bit and 64-bit datapath.

DLLP structure



Order of Arrival

0100b = InitFC1-P
 0101b = InitFC1-NP
 0110b = InitFC1-Cpl
 1100b = InitFC2-P
 1101b = InitFC2-NP
 1110b = InitFC2-Cpl
 1000b = UpdateFC-P
 1001b = UpdateFC-NP
 1010b = UpdateFC-Cpl



- DLLP Type = {tl_tx/rx_packet_type (2 bits), tl_tx/rx_update_type (2 bits), 1'b0, virtual channel number (3 bits)}
- R are reserved bytes and not considered here
- HdrFC is forwarded to TL via *tl_tx_hdr_credit*
- DataFC is forwarded to TL via *tl_tx_data_credit*

- **DLLP RX Handler**

- The DLL has a set of registers large enough to store a DLLP. Received data is shifted into these registers.
- If SDP is detected, extract the DLLP from the register with alignment.
- The data of DLLP is then extracted and CRC is checked.
- If CRC check failed, discard the packet.
- If CRC check passed, forward the data to Transaction Layer with `tl_rx_dllp_valid` asserted.
- In case of ACK and NAK.
 - On ACK, release all unacknowledged TLPs in the retry buffer (no more than two as it is the max size of retry buffer) at and below of the sequence number stored in ACK. Update `ACKD_SEQ` to `Ack_Seq_Num`.
 - On NAK, retransmit all unacknowledged TLPs.

- **DLLP TX Handler**

- DLL receives `tl_tx_dllp_valid` from TL to initiate transmission of a DLLP.
- Needed data to form a DLLP is also taken from TL. CRC is calculated and added afterward.
- If LCRC check for a TLP failed, send NAK with the sequence number of the erroneous packet.
- If LCRC check for a TLP passed, send ACK with the sequence number of the error-free packet.

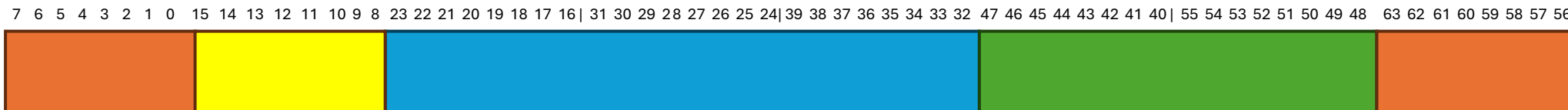
ACK/NAK structure



00000000b = ACK

00010000b = NAK

Order of Arrival



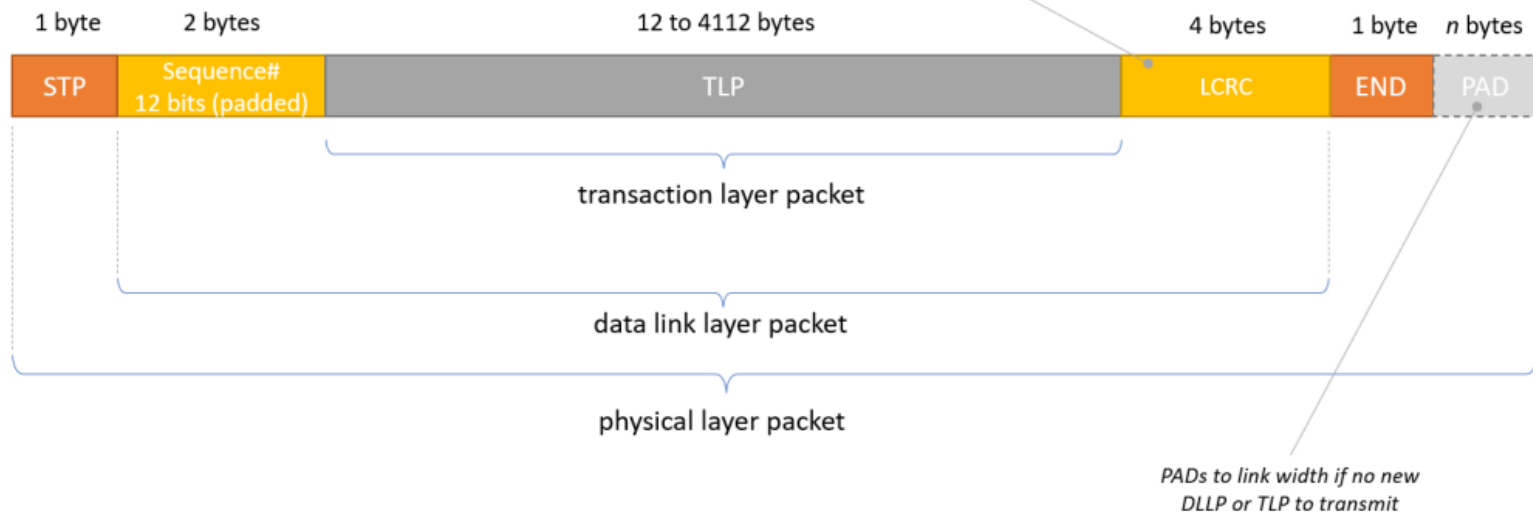
TLP structure

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

Initial value is FFFFFFFFh

DLLP bytes fed in LSb first

Bytes of result inverted and bit reversed



STP = 1111 1011

END = 1111 1101

EDB = 1111 1110

- TLP TX Handler
 - TL indicates the start and end of TLP payload.
 - DLL stores the TLP payload in RAM.
 - Each TLP is assigned to a 12-bit sequence number.
 - TLP is protected by a 32-bit LCRC.
 - The polynomial used is defined in the previous slide.
 - Result of the calculation is complemented and bits of each byte are stored in the reversed order.
 - Priority of transmissions: TLP/DLLP in progress, NAK, ACK, FC DLLP, Retry Buffer re-transmissions, TLP transmission from transaction layer, other DLLP transmission.

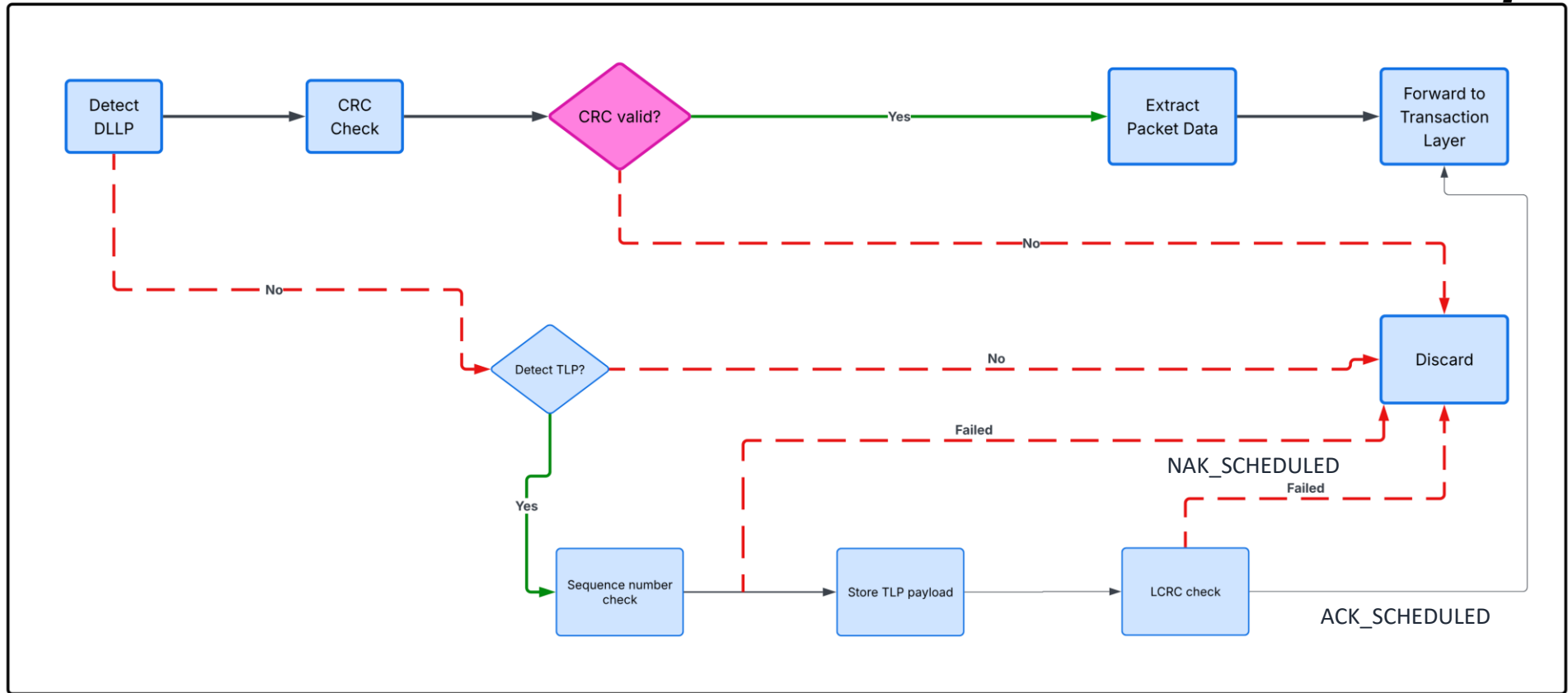
- **TLP RX Handler**
 - Received TLP payload is stored in RAM, waiting for integrity check.
 - NEXT_RCV_SEQ stores the expected sequence number for the next TLP.
 - NAK_SCHEDULED is asserted when a NAK is scheduled.
 - LCRC not equal, TLP is corrupt, NAK_SCHEDULED is asserted.
 - If TLP sequence number is not equal to the expected value
 - $(\text{NEXT_RCV_SEQ} - \text{TLP Sequence Number}) \bmod 4096 \leq 2048$, TLP is a duplicate, an ACK is scheduled.
 - Otherwise, the TLP is out of sequence. NAK_SCHEDULED is asserted.
 - AckNak_LATENCY_TIMER counts time since an Ack or Nak DLLP was scheduled for transmission.
 - If TLP sequence number is equal to the expected value, control data is removed, TLP is forwarded to TL.
 - DLL denotes the start and end of TLP
 - NEXT_RCV_SEQ is incremented
 - If set, the NAK_SCHEDULED flag is cleared
 - ACK is scheduled
 - ACK is sent when: In DL_Active, ACK is scheduled, AckNak_LATENCY_TIMER exceeds value in Table 3-5, NAK_SCHEDULED is cleared.
 - Nullification of TLP with EDB is not supported in this first version of DLL.

Retry Buffer

- Retry Buffer
 - 2-bit replay number counts the number of times the Retry Buffer has been re-transmitted.
 - If replay number roll over from 11b to 00b, signal Physical layer to retrain the link.
 - Replay timer counts the time since last transmission or retransmission.
 - First version would offer a retry buffer with the size of one.
 - Later version might move to a retry buffer with the size of two, implemented with a ping-pong scheme.

Simple RX flow

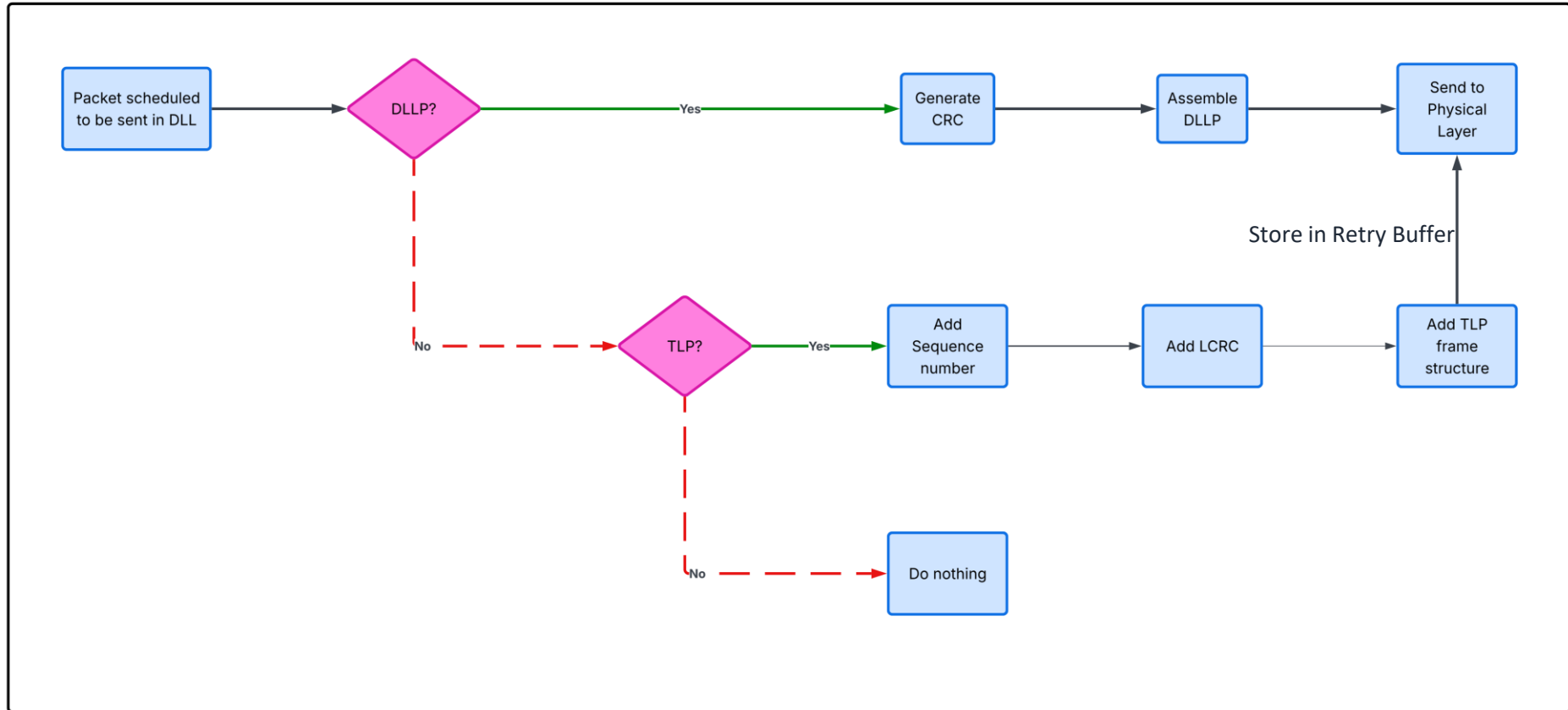
Receiver Data Flow: DLLP to Transaction Layer



Also, DLLP Update must be scheduled for sending at regular intervals. This defaults to 30µs (-0%/+50%) but can be configured to be 120µs

Simple TX flow

Transmitter Data Flow: DLLP to Physical Layer



Contact

Luong Vinh Hai, B.Sc.

Cologne Chip AG

Eintrachtstr. 113

50668 Köln

